

IncidentFlow

Architectures & Mécanismes de Persistance des Données

Spécification Technique de Persistance JPA / Hibernate

Modélisation et Spécifications des Entités

Utilisateurs, Rôles, Permissions & Workflows Dynamiques

Auteur :

Équipe de Développement IncidentFlow
NetMar Technologies

Composants analysés :

Spring Data JPA / Hibernate 6
PostgreSQL 16 & Cache Redis

Table des matières

1	Introduction et Vue d'Ensemble de l'Architecture	2
2	Modèle de Données et Relations JPA	2
2.1	Diagramme Entité-Association (ERD)	2
3	Persistance des Utilisateurs, Rôles et Permissions (RBAC)	3
3.1	Entités <code>Role</code> et <code>Permission</code>	3
3.1.1	Points Clés de Persistance RBAC :	3
3.2	Entité <code>User</code>	3
4	Persistance des Workflows, États et Transitions	4
4.1	Entité <code>Workflow</code>	4
4.2	Entités <code>WorkflowState</code> et <code>WorkflowTransition</code>	5
5	Gestion du Cycle de Vie JPA & Résolution de l'Erreur <i>Detached Entity</i>	5
5.1	Analyse du Problème de Persistance (<i>Detached Entity Passed to Persist</i>)	5
5.2	Solution Implémentée dans <code>WorkflowService</code>	6
6	Flux de Persistance Frontend-Backend	7
7	Conclusion et Bonnes Pratiques Retenues	8

1 Introduction et Vue d'Ensemble de l'Architecture

L'application **IncidentFlow** est une plateforme de gestion dynamique d'incidents IT et médicaux s'appuyant sur un moteur de *State Machine* personnalisable. La robustesse et l'intégrité de la plateforme reposent sur la couche de persistance de données gérée par **Spring Data JPA** et **Hibernate ORM 6**, couplée à une base de données relationnelle **PostgreSQL 16** et à une couche de cache distribué **Redis**.

Ce document spécifie en détail la modélisation, le comportement transactionnel et les stratégies d'association JPA pour les trois piliers fondamentaux du système :

- **La Gestion des Utilisateurs (User)** : Authentification, profils et affectations organisationnelles.
- **La Matrice de Sécurité RBAC (Role & Permission)** : Attribution dynamique des droits système et contrôle d'accès fin.
- **Les Workflows Dynamiques (Workflow, WorkflowState, WorkflowTransition)** : Définition visuelle des cycles de vie d'incidents, gestion des contraintes de transition et des rôles requis.

2 Modèle de Données et Relations JPA

2.1 Diagramme Entité-Association (ERD)

Le schéma relationnel ci-dessous illustre la structure des tables PostgreSQL et les liens de clés étrangères reliant les utilisateurs, les rôles, les permissions et les composants de workflows.

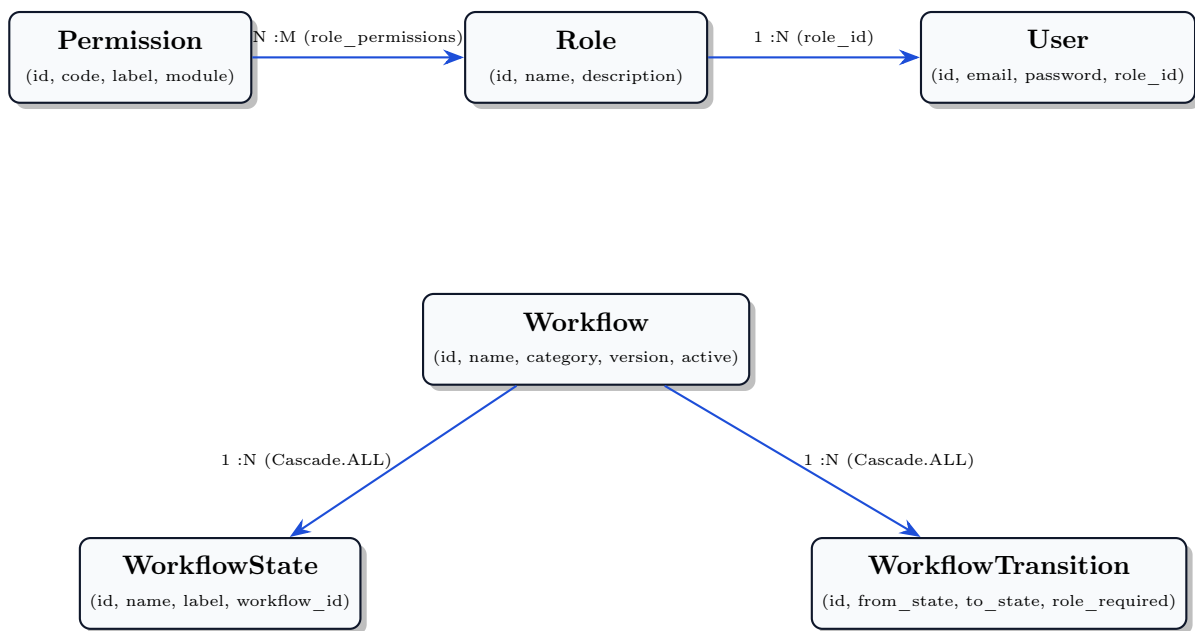


FIGURE 1 – Diagramme Entité-Association de la persistance IncidentFlow.

3 Persistance des Utilisateurs, Rôles et Permissions (RBAC)

3.1 Entités Role et Permission

Le système RBAC (*Role-Based Access Control*) s'appuie sur une relation Many-to-Many entre les entités Role et Permission.

```

1 @Entity
2 @Table(name = "roles")
3 @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
4 public class Role implements Serializable {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @Column(nullable = false, unique = true)
11    private String name;
12
13    private String description;
14
15    @ManyToMany(fetch = FetchType.EAGER)
16    @JoinTable(
17        name = "role_permissions",
18        joinColumns = @JoinColumn(name = "role_id"),
19        inverseJoinColumns = @JoinColumn(name = "permission_id")
20    )
21    @Builder.Default
22    private Set<Permission> permissions = new HashSet<>();
23 }

```

Listing 1 – Mapping JPA de l'entité Role.

3.1.1 Points Clés de Persistance RBAC :

1. **Chargement Immédiat** (`FetchType.EAGER`) : Les permissions associées à un rôle sont chargées immédiatement lors de l'authentification pour alimenter le contexte de sécurité Spring Security sans requêtes N+1.
2. **Jointure** `role_permissions` : La table d'association conserve la correspondance exacte entre les identifiants de rôles et de privilèges.

3.2 Entité User

L'utilisateur possède une association Many-to-One vers la table des rôles :

```

1 @Entity
2 @Table(name = "users")
3 public class User implements Serializable {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, unique = true)
10    private String email;
11
12    @Column(nullable = false)
13    private String password;

```

```

14
15     @ManyToOne(optional = false, fetch = FetchType.EAGER)
16     @JoinColumn(name = "role_id", nullable = false)
17     private Role role;
18 }

```

Listing 2 – Extrait de l'entité User.

4 Persistance des Workflows, États et Transitions

La gestion des workflows dynamiques est la fonctionnalité cœur de l'application. Un workflow est composé d'un ensemble d'états (`WorkflowState`) et de règles de transition autorisées (`WorkflowTransition`).

4.1 Entité Workflow

L'entité principale `Workflow` agrège ses états et transitions via une cascade complète et une gestion stricte des entités orphelines (`orphanRemoval = true`).

```

1  @Entity
2  @Table(name = "workflows")
3  @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
4  public class Workflow implements Serializable {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9
10     @Column(nullable = false)
11     private String name;
12
13     @Column(nullable = false)
14     private String category;
15
16     @Column(nullable = false)
17     @Builder.Default
18     private int version = 1;
19
20     @Column(nullable = false)
21     @Builder.Default
22     private boolean active = true;
23
24     @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL,
25               orphanRemoval = true, fetch = FetchType.EAGER)
26     @Builder.Default
27     private List<WorkflowState> states = new ArrayList<>();
28
29     @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL,
30               orphanRemoval = true, fetch = FetchType.EAGER)
31     @Builder.Default
32     private List<WorkflowTransition> transitions = new ArrayList<>();
33 }

```

Listing 3 – Entité Workflow et ses collections associées.

4.2 Entités WorkflowState et WorkflowTransition

Chaque transition contient désormais un champ `roleRequired` permettant de restreindre la réalisation d'un changement d'état à un rôle spécifique.

```

1 @Entity
2 @Table(name = "workflow_transitions")
3 @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
4 public class WorkflowTransition implements Serializable {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @Column(name = "from_state", nullable = false)
11    private String fromState;
12
13    @Column(name = "to_state", nullable = false)
14    private String toState;
15
16    @Column(name = "role_required")
17    private String roleRequired; // Exemple: "Administrateur", "
        Responsable", null
18
19    @Column(name = "requires_comment", nullable = false)
20    @Builder.Default
21    private boolean requiresComment = false;
22
23    @ManyToOne(optional = false)
24    @JoinColumn(name = "workflow_id", nullable = false)
25    @JsonIgnore
26    private Workflow workflow;
27 }

```

Listing 4 – Entite WorkflowTransition avec restriction de role.

5 Gestion du Cycle de Vie JPA & Résolution de l'Erreur *Detached Entity*

5.1 Analyse du Problème de Persistance (*Detached Entity Passed to Persist*)

Lors de la mise à jour d'un workflow existant depuis l'interface Web (Frontend React), le contrôleur reçoit un objet JSON désérialisé contenant des états et des transitions ayant des identifiants (`id`) déjà générés.

Lorsque l'on tente d'affecter directement cette liste à une entité managée par Hibernate :

1. L'instance désérialisée n'appartient pas au contexte de persistance (*Persistence Context*) courant.
2. Hibernate détecte un identifiant non nul sur des objets enfants et interprète l'objet comme une entité détachée (*Detached Entity*).
3. L'appel à la méthode `persist()` au travers de la cascade `CascadeType.ALL` déclenche l'exception :
`PersistentObjectException: detached entity passed to persist.`

5.2 Solution Implémentée dans WorkflowService

Pour résoudre ce problème de manière définitive et garantir l'intégrité référentielle, le service procède par mise à jour en place sur la collection managée (*In-Place Update*) ou par versioning automatique si des incidents sont déjà rattachés au workflow.

```

1  @Transactional
2  @CacheEvict(value = {"workflows", "workflows-category"}, allEntries =
   true)
3  public Workflow saveWorkflow(Workflow workflow) {
4      validateWorkflowGraph(workflow);
5
6      if (workflow.getId() != null) {
7          boolean hasLinkedIncidents = incidentRepository.
            existsByWorkflowId(workflow.getId());
8
9          // Cas 1 : Des incidents sont liés -> Creation d'une nouvelle
            version (Versioning)
10         if (hasLinkedIncidents) {
11             Workflow original = workflowRepository.findById(workflow.
                getId())
12                 .orElseThrow(() -> new ResourceNotFoundException("
                    Workflow original non trouve"));
13             original.setActive(false);
14             workflowRepository.save(original);
15
16             Workflow newVersion = new Workflow();
17             newVersion.setName(workflow.getName());
18             newVersion.setCategory(workflow.getCategory());
19             newVersion.setVersion(original.getVersion() + 1);
20             newVersion.setActive(true);
21
22             // Re-instanciation propre sans ID pour eviter l'etat detache
23             if (workflow.getStates() != null) {
24                 for (WorkflowState state : workflow.getStates()) {
25                     newVersion.getStates().add(WorkflowState.builder()
26                         .name(state.getName()).label(state.getLabel()
27                             )
28                         .colorClass(state.getColorClass()).active(
29                             state.isActive())
30                         .workflow(newVersion).build());
31                 }
32             }
33             if (workflow.getTransitions() != null) {
34                 for (WorkflowTransition transition : workflow.
35                     getTransitions()) {
36                     newVersion.getTransitions().add(WorkflowTransition.
37                         builder()
38                             .fromState(transition.getFromState()).toState
39                             (transition.getToState())
40                             .roleRequired(transition.getRoleRequired())
41                             .requiresComment(transition.isRequiresComment
42                                 ())
43                             .workflow(newVersion).build());
44                 }
45             }
46             return workflowRepository.save(newVersion);
47         }
48     }
49 }

```

```

43      // Cas 2 : Aucun incident lie -> Nettoyage et mise a jour en
44      place
45      Workflow existing = workflowRepository.findById(workflow.getId())
46                          .orElseGet(() -> workflow);
47
48      if (existing != workflow) {
49          existing.setName(workflow.getName());
50          existing.setCategory(workflow.getCategory());
51          existing.setActive(workflow.isActive());
52
53          // Nettoyage de la collection managée (declenche
54          orphanRemoval)
55          existing.getStates().clear();
56          if (workflow.getStates() != null) {
57              for (WorkflowState state : workflow.getStates()) {
58                  existing.getStates().add(WorkflowState.builder()
59                      .name(state.getName()).label(state.getLabel())
60                      .colorClass(state.getColorClass()).active(
61                          state.isActive())
62                      .workflow(existing).build());
63              }
64          }
65
66          existing.getTransitions().clear();
67          if (workflow.getTransitions() != null) {
68              for (WorkflowTransition transition : workflow.
69                  getTransitions()) {
70                  existing.getTransitions().add(WorkflowTransition.
71                      builder()
72                      .fromState(transition.getFromState()).toState(
73                          transition.getState())
74                      .roleRequired(transition.getRoleRequired())
75                      .requiresComment(transition.isRequiresComment())
76                      .workflow(existing).build());
77              }
78          }
79          return workflowRepository.save(existing);
80      }
81      return workflowRepository.save(workflow);
82  }

```

Listing 5 – Algorithme de sauvegarde sécurisée dans WorkflowService.java

6 Flux de Persistance Frontend-Backend

Le diagramme de séquence suivant résume le parcours de persistance lors de la modification du rôle autorisé d'une transition depuis l'interface utilisateur React.

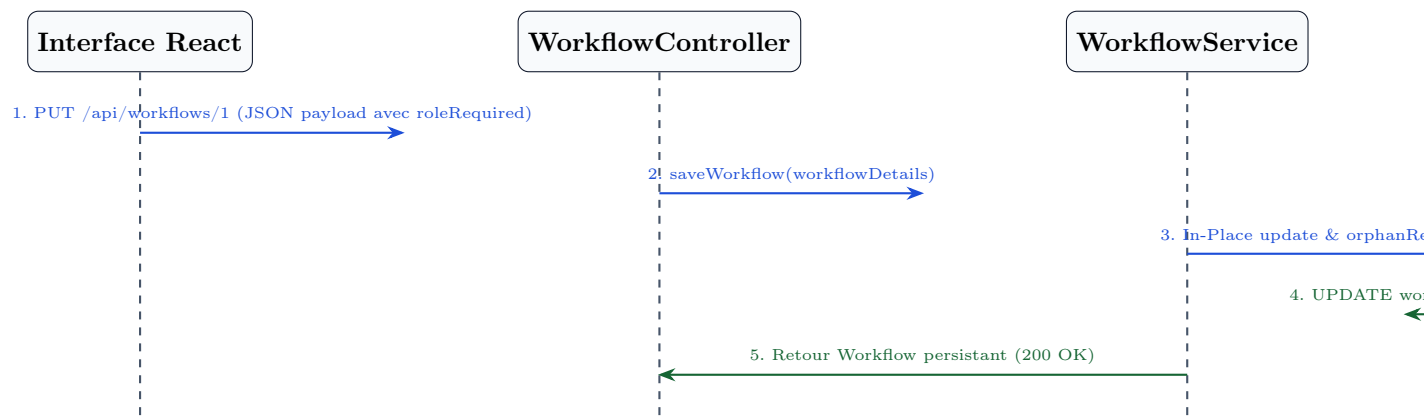


FIGURE 2 – Diagramme de Séquence de la persistance d’une transition modifiée.

7 Conclusion et Bonnes Pratiques Retenues

La stratégie de persistance mise en œuvre pour les entités **User**, **Role** et **Workflow** au sein d’IncidentFlow garantit :

- **Intégrité des données** : Aucune incohérence d’état n’est possible grâce aux validations graphiques par parcours en profondeur (DFS) et au nettoyage automatique des sous-collections via `orphanRemoval`.
- **Gestion fluide des versions** : En présence d’incidents associés, l’historique n’est jamais corrompu car les anciens workflows sont archivés et une nouvelle version active est créée.
- **Sécurité granulaire RBAC** : La restriction dynamique des transitions par rôle (`roleRequired`) est directement enregistrée et validée au niveau de la couche service lors de chaque passage d’état d’un incident.